GLOBALRAIN

**Practices for Secure Software Report**

## Table of Contents

DOCUMENT REVISION HISTORY ..... 3
CLIENT ..... 3
INSTRUCTIONS ..... 3
DEVELOPER ..... 4
1. ALGORITHM CIPHER ..... 4
2. CERTIFICATE GENERATION ..... 4
3. DEPLOY CIPHER ..... 4
4. SECURE COMMUNICATIONS ..... 4
5. SECONDARY TESTING ..... 4
6. FUNCTIONAL TESTING ..... 4
7. SUMMARY ..... 4
8. INDUSTRY STANDARD BEST PRACTICES ..... 4

**Document Revision History**

| Version | Date | Author | Comments |
|---|---|---|---|
| 1.0 | 2/14/2026 | Alexis Lewis | |

**Client**

**Instructions**

Submit this completed practices for secure software report. Replace the bracketed text with the relevant information. You must document your process for writing secure communications and refactoring code that complies with software security testing protocols.

- Respond to the steps outlined below and include your findings.
- Respond using your own words. You may also choose to include images or supporting materials. If you include them, make certain to insert them in all the relevant locations in the document.
- Refer to the Project Two Guidelines and Rubric for more detailed instructions about each section of the template.

**Developer**
Alexis Lewis

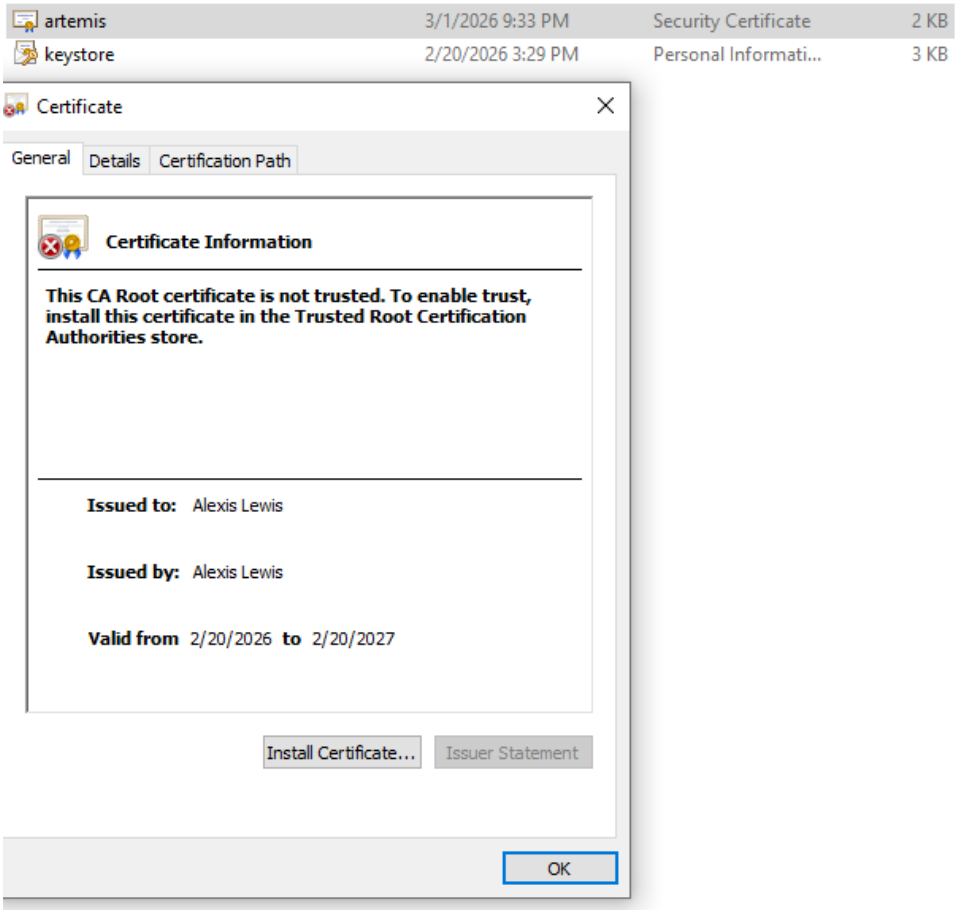
## 1. Algorithm Cipher

To safeguard Artemis Financial's public web interface and sensitive client financial data, it is recommended to implement HTTPS with Transport Layer Security (TLS), utilizing AES-256 for encryption and SHA-256 for integrity verification. AES (Advanced Encryption Standard) is a symmetric block cipher that encrypts 128-bit data blocks with a 256-bit key, offering robust resistance to brute-force attacks and conforming to current industry standards. For file verification, SHA-256, a cryptographic hash function from the SHA-2 family, produces a fixed 256-bit checksum. Hash functions, unlike encryption, are one-way operations designed to verify data integrity and ensure that transmitted data remains unaltered. The combined use of AES-256 and SHA-256 ensures both confidentiality and integrity in secure communications.

TLS employs a hybrid cryptographic model that integrates both asymmetric and symmetric encryption. During the TLS handshake, asymmetric algorithms such as RSA-2048 or elliptic curve cryptography utilize public and private keys to securely exchange a session key. After the session key is established, symmetric encryption (AES) is applied for efficient data transmission. Cryptographically secure random numbers generate session keys and nonces, mitigating the risk of prediction or replay attacks. Previous algorithms such as DES, MD5, and SHA-1 are now considered insecure due to insufficient key lengths and susceptibility to collision attacks. Current standards prioritize AES and SHA-256, making them the most suitable and effective options for Artemis Financial's secure software modernization.

## 2. Certificate Generation

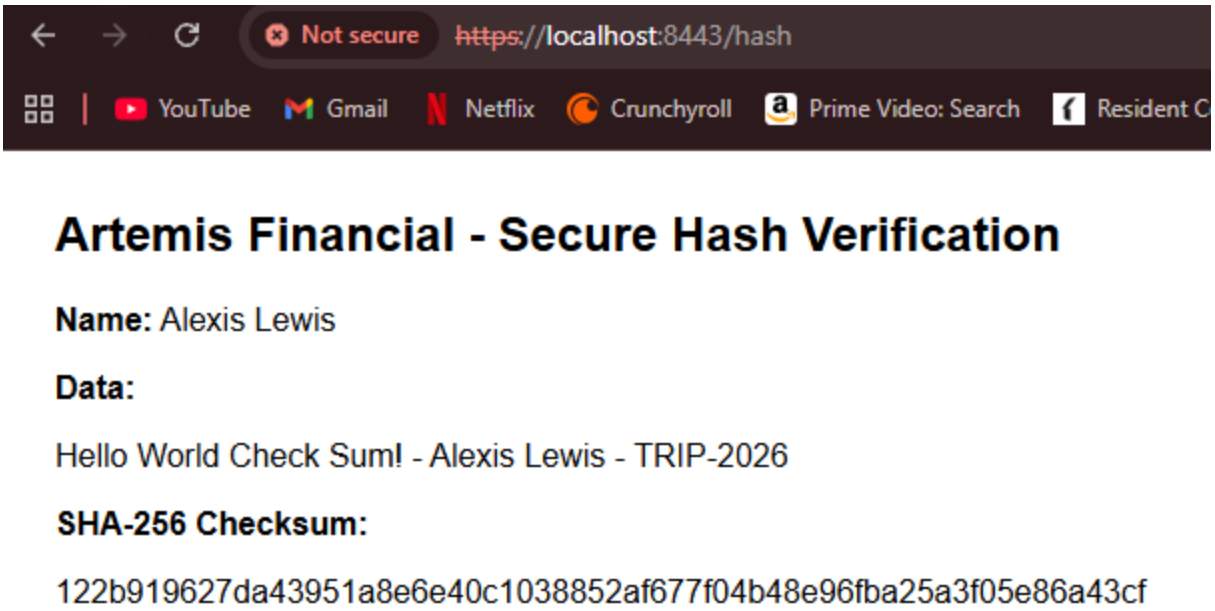
Insert a screenshot below of the CER file.




A public certificate (.cer) was exported from the PKCS#12 keystore using the Java keytool utility. The certificate contains the public key required for TLS encryption and may be distributed to facilitate trust in secure communications.

### 3. Deploy Cipher

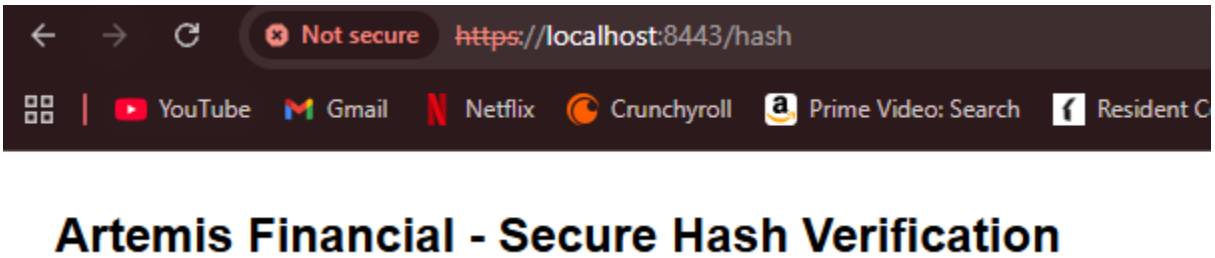
Insert a screenshot below of the checksum verification.




SHA-256 was implemented as the hashing algorithm to verify data integrity. The resulting checksum confirms that the input data remains unaltered and can be authenticated with a high degree of security.

**4. Secure Communications**

Insert a screenshot below of the web browser that shows a secure webpage.




Secure communication was verified by accessing the application through HTTPS (https://localhost:8443). Despite the browser indicating “Not Secure” because of a self-signed development certificate, the connection remains encrypted using TLS. The SHA-256 hash is generated securely and presented via the HTTPS endpoint.

**5. Secondary Testing**

Insert screenshots below of the refactored code executed without errors and the dependency-check report.

## Summary

Summary of Vulnerable Dependencies (click to show all)

| Dependency | Vulnerability IDs | Package | Highest Severity | CVE Count | Confidence | Evidence Count |
|---|---|---|---|---|---|---|
| hibernate-validator-6.0.18.Final.jar | cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:* <br> cpe:2.3:a:redhat:hibernate_validator:6.0.18:*:*:*:*:*:*:* | pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final | MEDIUM | 3 | Highest | 32 |
| jackson-databind-2.10.2.jar | cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:* <br> cpe:2.3:a:fasterxml:jackson-modules-java8:2.10.2:*:*:*:*:*:*:* | pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2 | HIGH | 6 | Highest | 39 |
| json-path-2.4.0.jar | cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*:*:* | pkg:maven/com.jayway.jsonpath/json-path@2.4.0 | MEDIUM | 1 | Highest | 33 |
| json-smart-2.3.jar | cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*:*:* <br> cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*:*:*:*:* | pkg:maven/net.minidev/json-smart@2.3 | HIGH | 2 | Highest | 45 |
| log4j-api-2.12.1.jar | cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:* | pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1 | MEDIUM | 2 | Highest | 42 |
| logback-core-1.2.3.jar | cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:* | pkg:maven/ch.qos.logback/logback-core@1.2.3 | HIGH | 2 | Highest | 31 |
| snakeyaml-1.25.jar | cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:* | pkg:maven/org.yaml/snakeyaml@1.25 | CRITICAL | 8 | Highest | 44 |
| spring-boot-2.2.4.RELEASE.jar | cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:* | pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE | CRITICAL | 3 | Highest | 39 |
| spring-core-5.2.3.RELEASE.jar | cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* <br> cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* <br> cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:* | pkg:maven/org.springframework/spring-core@5.2.3.RELEASE | CRITICAL* | 12 | Highest | 36 |
| spring-hateoas-1.0.3.RELEASE.jar | cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*:*:*:*:* | pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE | MEDIUM | 1 | Highest | 43 |
| spring-web-5.2.3.RELEASE.jar | cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:* <br> cpe:2.3:a:springsource:spring_framework:5.2.3:release:*:*:*:*:*:* <br> cpe:2.3:a:vmware:spring_framework:5.2.3:release:*:*:*:*:*:* | pkg:maven/org.springframework/spring-web@5.2.3.RELEASE | CRITICAL* | 13 | Highest | 34 |
| tomcat-embed-core-9.0.30.jar | cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* <br> cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:* | pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30 | CRITICAL* | 47 | Highest | 30 |
| tomcat-embed-websocket-9.0.30.jar | cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:* <br> cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*:*:*:*:*:* | pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30 | CRITICAL* | 48 | Highest | 30 |

```
[INFO] Finished Known Exploited Vulnerability Analyzer (0 seconds)
[INFO] Finished Dependency Bundling Analyzer (0 seconds)
[INFO] Finished Unused Suppression Rule Analyzer (0 seconds)
[INFO] Analysis Complete (3 seconds)
[INFO] Writing HTML report to: C:\Users\Alexis Williams\OneDrive\Desktop\Schoolwork\CS-305\CS 305 Project Two Code Base\ssl-server_student\target\dependency-check-report.html
[WARNING]

One or more dependencies were identified with known vulnerabilities in ssl-server:

hibernate-validator-6.0.18.Final.jar (pkg:maven/org.hibernate.validator/hibernate-validator@6.0.18.Final, cpe:2.3:a:hibernate:hibernate-validator:6.0.18:*:*:*:*:*:*:*, cpe:2.3:a:redhat
jackson-databind-2.10.2.jar (pkg:maven/com.fasterxml.jackson.core/jackson-databind@2.10.2, cpe:2.3:a:fasterxml:jackson-databind:2.10.2:*:*:*:*:*:*:*, cpe:2.3:a:fasterxml:jackson-module
json-path-2.4.0.jar (pkg:maven/com.jayway.jsonpath/json-path@2.4.0, cpe:2.3:a:json-path:jayway_jsonpath:2.4.0:*:*:*:*:*:*:*) : CVE-2023-51074
json-smart-2.3.jar (pkg:maven/net.minidev/json-smart@2.3, cpe:2.3:a:json-smart_project:json-smart:2.3:*:*:*:*:*:*:*, cpe:2.3:a:json-smart_project:json-smart-v2:2.3:*:*:*:*:*:*:*) : CVE
log4j-api-2.12.1.jar (pkg:maven/org.apache.logging.log4j/log4j-api@2.12.1, cpe:2.3:a:apache:log4j:2.12.1:*:*:*:*:*:*:*) : CVE-2025-68161, CVE-2020-9488
logback-core-1.2.3.jar (pkg:maven/ch.qos.logback/logback-core@1.2.3, cpe:2.3:a:qos:logback:1.2.3:*:*:*:*:*:*:*) : CVE-2023-6378, CVE-2021-42550
snakeyaml-1.25.jar (pkg:maven/org.yaml/snakeyaml@1.25, cpe:2.3:a:snakeyaml_project:snakeyaml:1.25:*:*:*:*:*:*:*) : CVE-2022-1471, CVE-2017-18640, CVE-2022-25857, CVE-2022-38749, CVE-26
spring-boot-2.2.4.RELEASE.jar (pkg:maven/org.springframework.boot/spring-boot@2.2.4.RELEASE, cpe:2.3:a:vmware:spring_boot:2.2.4:release:*:*:*:*:*:*) : CVE-2023-20873, CVE-2022-27772, (
spring-core-5.2.3.RELEASE.jar (pkg:maven/org.springframework/spring-core@5.2.3.RELEASE, cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*, cpe:2.3:a:springsource:sp
spring-hateoas-1.0.3.RELEASE.jar (pkg:maven/org.springframework.hateoas/spring-hateoas@1.0.3.RELEASE, cpe:2.3:a:vmware:spring_hateoas:1.0.3:release:*:*:*:*:*:*) : CVE-2023-34036
spring-web-5.2.3.RELEASE.jar (pkg:maven/org.springframework/spring-web@5.2.3.RELEASE, cpe:2.3:a:pivotal_software:spring_framework:5.2.3:release:*:*:*:*:*:*, cpe:2.3:a:springsource:spri
tomcat-embed-core-9.0.30.jar (pkg:maven/org.apache.tomcat.embed/tomcat-embed-core@9.0.30, cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:*, cpe:2.3:a:apache_tomcat:apache_tomcat:9.0.30:*:*
tomcat-embed-websocket-9.0.30.jar (pkg:maven/org.apache.tomcat.embed/tomcat-embed-websocket@9.0.30, cpe:2.3:a:apache:tomcat:9.0.30:*:*:*:*:*:*:*, cpe:2.3:a:apache_tomcat:apache_tomcat:


See the dependency-check report for more details.


[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  49.365 s
[INFO] Finished at: 2026-03-01T22:06:54-05:00
[INFO] ------------------------------------------------------------------------
```

Secondary static analysis was conducted using the OWASP Dependency-Check Maven plugin to evaluate project dependencies for known security vulnerabilities. This scan was performed following the implementation of secure communication enhancements, including TLS configuration and SHA-256 checksum functionality, to verify that no additional vulnerabilities were introduced during the refactoring process.

The generated report identified several known vulnerabilities within existing framework dependencies, such as outdated Spring Boot and Tomcat components. These vulnerabilities originated from the base project configuration and were not introduced by the secure communication enhancements. The review confirmed that the refactored code did not incorporate new insecure libraries or compromise the application's security posture. The following screenshots demonstrate the successful execution of the refactored application and the output of the dependency-check report.

### 6. Functional Testing

Insert a screenshot below of the refactored code executed without errors.

```
SslServerApplication [Java Application] C:\Program Files\Eclipse Adoptium\jdk-17.0.17.10-hotspot\bin\javaw.exe  (Mar 1, 2026, 8:52:53 PM elapsed 0:00:11) [pid: 22656]

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.4.RELEASE)

2026-03-01 20:52:56.882  INFO 22656 --- [           main] com.snhu.sslserver.SslServerApplication  : Starting SslServerApplication on Risky-AlexisPC with PID 22656 (started by Alexis Williams in
2026-03-01 20:52:56.883  INFO 22656 --- [           main] com.snhu.sslserver.SslServerApplication  : No active profile set, falling back to default profiles: default
2026-03-01 20:52:57.761  INFO 22656 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8443 (https)
2026-03-01 20:52:57.770  INFO 22656 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2026-03-01 20:52:57.770  INFO 22656 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.30]
2026-03-01 20:52:57.852  INFO 22656 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2026-03-01 20:52:57.852  INFO 22656 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed in 937 ms
2026-03-01 20:52:58.275  INFO 22656 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor'
2026-03-01 20:52:58.813  INFO 22656 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8443 (https) with context path ''
2026-03-01 20:52:58.816  INFO 22656 --- [           main] com.snhu.sslserver.SslServerApplication  : Started SslServerApplication in 2.182 seconds (JVM running for 4.376)
```

Functional testing demonstrated that the secure hash endpoint reliably generates a SHA-256 checksum when supplied with input data. The application operates without runtime errors and consistently produces identical hash outputs.

## 7. Summary

The Artemis Financial application underwent refactoring to enhance secure communications and comply with contemporary security standards. The vulnerability assessment identified two primary security gaps: unencrypted communication and the lack of integrity verification for transferred data. To address these issues, HTTPS was implemented using TLS with a PKCS#12 keystore to encrypt data in transit, and a SHA-256 hashing function was introduced to enable checksum verification for file integrity. These measures strengthen confidentiality and integrity by safeguarding data during transmission and ensuring that transferred data remains unaltered.

Subsequent static analysis was performed using OWASP Dependency-Check to evaluate third-party libraries for known vulnerabilities. The findings were reviewed to ensure that the secure communication refactoring did not introduce additional security risks. This iterative testing approach supports secure software development by validating modifications, identifying potential weaknesses, and ensuring adherence to security testing protocols prior to deployment. By integrating encryption, hashing, and static analysis, the application now employs multiple safeguards to protect sensitive financial data.

## 8. Industry Standard Best Practices

Industry standard best practices were implemented to improve and maintain the security of the software application. All data transmitted through the public web interface is protected with TLS encryption, ensuring the confidentiality of client financial information. Strong cryptographic standards such as AES (within TLS) and SHA-256 hashing were selected, while outdated algorithms like MD5 and SHA-1 were avoided. For key management, a PKCS#12 keystore was utilized instead of hardcoded credentials. Static vulnerability scans using OWASP Dependency-Check were conducted to identify known risks in third-party libraries and to ensure that no new vulnerabilities were introduced during refactoring.

Adherence to industry standard secure coding practices helps the company avoid data breaches, regulatory penalties, and reputational harm. Secure encryption, integrity verification, and continuous vulnerability assessments protect sensitive customer financial data and maintain client trust. Integrating security into the development process from the outset demonstrates that security is a shared responsibility and supports the long-term stability of Artemis Financial.